Interpolation: The Ideas Behind the Code

Lines, polynomials, splines, and shape preservation

Sharareh Sayyad

Washington State University

October 12, 2026

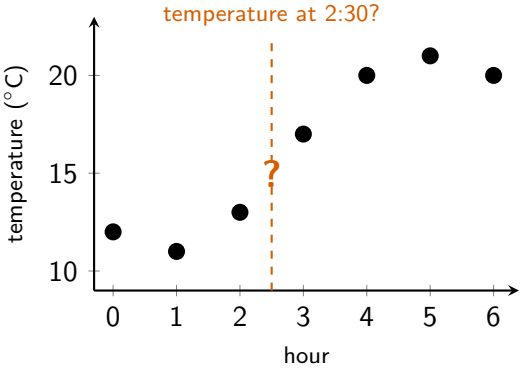
Why do we need interpolation?

We often know a quantity **only at a few points**, but need it **in between**.

- **Measurements:** a sensor reads once per hour.
- **Tables:** e.g. steam tables.
- **Expensive functions:** hours of computing per point.
- **Other methods:** integration, derivatives, ODE solvers build on it.

The task

given (x_i, y_i), find p with $p(x_i) = y_i$



Hourly readings. What was the temperature at 2:30? The answer depends on **how we connect the dots**: that is this lecture.

Idea 1: connect the dots with straight lines

On each interval $[x_i, x_{i+1}]$:

$$p(x) = (1 - t)\, y_i + t\, y_{i+1}$$

$$t = \frac{x - x_i}{x_{i+1} - x_i} \in [0, 1]$$

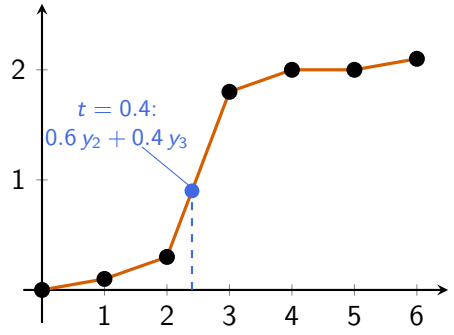
A **weighted average** of the two neighbours.

Good

Local: only 2 points matter. Never overshoots.
Error $\leq \frac{h^2}{8} \max |f''|$: halve h, error $\div 4$.

Not so good

A **corner** at every node.



Running example: 7 points that rise, then level off.
We use the same data for every method.

Idea 2: one polynomial through all the points

Degree $n \Rightarrow n+1$ coefficients

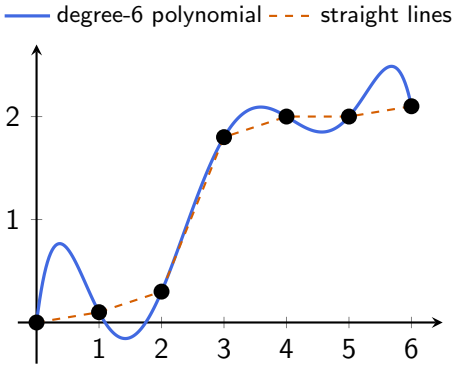
		deg.	coeff.
line	$a + bx$	1	2
quadratic	$a + bx + cx^2$	2	3
cubic	$a + bx + cx^2 + dx^3$	3	4

Each coefficient lets us hit one more point:
$n+1$ points $\Rightarrow$ one polynomial of degree n.

Good: one formula, smooth everywhere, no corners.

Not so good: it is global

Every point affects the **whole** curve.
To bend at $x = 3$, it must **wiggle** elsewhere.



Smooth, but it bulges above the last points and dips below zero.

Idea 3: piecewise polynomials

Use a **separate cubic on each interval**.

- **Local** like straight lines.
- **Smooth** like polynomials.

A cubic (degree 3) has 4 coefficients

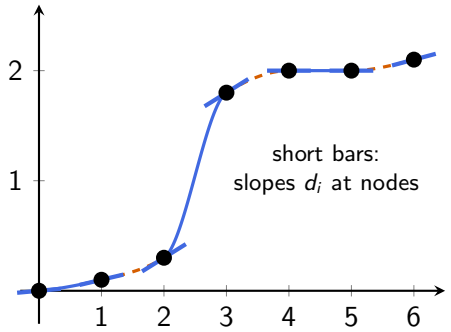
So each piece can meet 4 conditions:

2 values $y_i,\ y_{i+1}$ ← given by the data

2 slopes $d_i,\ d_{i+1}$ ← **we choose**

The big idea

Different cubic methods = **different rules for choosing the slopes** d_i.



Six pieces, alternating solid/dashed.
Neighbours share the value **and** the slope at each node.

Building one piece: 2 points + 2 slopes = 1 cubic

One interval $[0, 1]$, one cubic
$p(x) = a + bx + cx^2 + dx^3$.

We ask for 4 things:

values: $p(0) = y_0$, $p(1) = y_1$
slopes: $p'(0) = d_0$, $p'(1) = d_1$

Solving gives the 4 coefficients

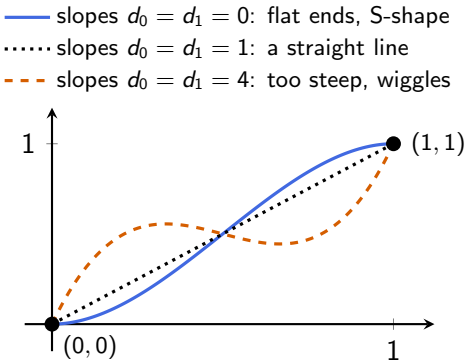
$a = y_0, \qquad b = d_0$

$c = 3(y_1 - y_0) - 2d_0 - d_1$

$d = 2(y_0 - y_1) + d_0 + d_1$

Message

Points: **fixed by the data**. Slopes: **our choice**, and they decide the shape.



Same two points, three slope choices, three cubics.
Neighbours share the slope at a node ⇒ no corners.

Rule A, cubic spline: match the second derivative too

New requirement: p'' **continuous** at every node.

Step 1: p'' at the ends of a piece

From slide 6, with $\delta_i = y_{i+1} - y_i$:

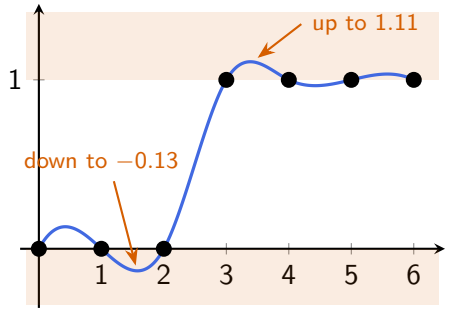
left: $p''(0) = \quad 6\delta_i - 4d_i - 2d_{i+1}$

right: $p''(1) = -6\delta_i + 2d_i + 4d_{i+1}$

Step 2: match at node i

$$d_{i-1} + 4\,d_i + d_{i+1} = 3\,(\delta_{i-1} + \delta_i)$$

$\Rightarrow$ all slopes are solved **together**.



The spline leaves the data range $[0, 1]$.

Why: the slopes are linked, so the jump also tilts its neighbours.

Rule B, shape-preserving (PCHIP): choose slopes locally

Look only at the two **secant slopes** next to node i: δ_{i-1} (left) and δ_i (right).

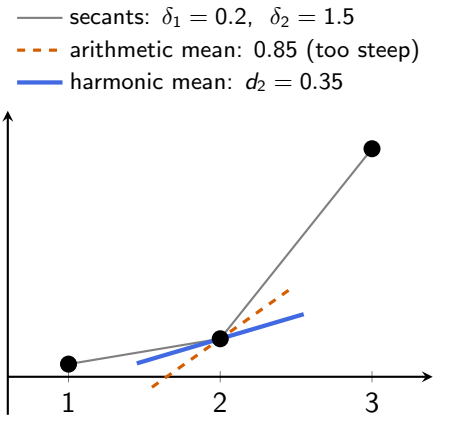
PCHIP rule

1. Data turn or go flat (different signs, or a 0) $\Rightarrow d_i = 0$

2. Otherwise, **harmonic mean**:

$$\frac{1}{d_i} = \frac{1}{2}\left(\frac{1}{\delta_{i-1}} + \frac{1}{\delta_i}\right)$$

The harmonic mean stays close to the **smaller** slope, so the curve does not shoot past the data.



Zoom on nodes 1–3 of the running example.

Shape-preserving: the result

What we gain

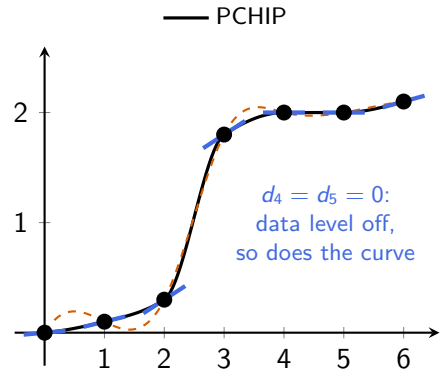
Rising data ⇒ rising curve.
No overshoot, no dips below the data.
Each slope uses only nearby points (local).

What we give up

p'' may jump at the nodes: only p and p' are continuous.

Akima is a cousin: a weighted average of δ_{i-1} and δ_i, also local, also avoids wiggles.

Use when the data must stay physical: positive, increasing, or bounded.



PCHIP never goes down and never exceeds 2.1.

Summary: how each method decides the curve

Method	**How it is decided**	**Smooth**	**Overshoot?**
Straight lines	2 neighbouring values	p	never
One polynomial	all points at once	all orders	often
Cubic spline	slopes from a global system	p, p', p''	sometimes
PCHIP / Akima	slopes from nearby secants	p, p'	no / rarely

Take-home messages

1. The trade-off is **smoothness vs locality**.
2. Piecewise cubics differ only in **how the slopes d_i are chosen**.
3. Smooth data, smooth curve wanted: **cubic spline**. Data must keep its shape: **PCHIP**.